گزارش کامل پروژه SimSiam

یادگیری خودنظارتی و طبقه‌بندی تصاویر CIFAR-10

گزارش فنی و آموزشی برای ارائه دانشگاهی

نسخه پروژه: ۱۰ نوامبر ۲۰۲۶

۱. معرفی پروژه

این پروژه یک پیاده‌سازی آموزشی و modular از روش **SimSiam** برای یادگیری خودنظارتی روی مجموعه‌داده CIFAR-10 است. هدف این است که مدل ابتدا بدون استفاده از برچسب‌ها ویژگی‌های عمومی تصاویر را یاد بگیرد و سپس با یک classifier خطی، کلاس تصویر را پیش‌بینی کند.

> **خلاصه نتیجه واقعی:** مدل SimSiam به همراه linear classifier روی ۱۰,۰۰۰ تصویر آزمایشی به دقت ۳۲٫۰۸٪ رسید. supervised baseline در همان آزمایش دقت ۷۰٫۰۱٪ داشت.

یادگیری خودنظارتی چیست؟

در یادگیری نظارت‌شده، هر تصویر به یک label نیاز دارد. در یادگیری خودنظارتی، هدف آموزشی از خود داده ساخته می‌شود. در SimSiam، دو نسخه تقویت‌شده از یک تصویر تولید می‌شود و مدل یاد می‌گیرد نمایش این دو نسخه به یکدیگر نزدیک باشد.

SimSiam چگونه کار می‌کند؟

1. از تصویر اصلی دو view با augmentation ساخته می‌شود.
2. هر view از encoder عبور می‌کند و به یک representation تبدیل می‌شود.
3. projector نمایش را به فضای embedding می‌برد.
4. predictor نمایش یک شاخه را پیش‌بینی می‌کند.
5. با negative cosine similarity دو شاخه مقایسه می‌شوند.
6. در شاخه هدف stop-gradient اعمال می‌شود.

Stop-gradient و تابع loss

در stop-gradient، خروجی هدف با `detach()` از graph گرادیان جدا می‌شود. بنابراین گرادیان فقط از مسیر prediction عبور می‌کند. این کار به جلوگیری از collapse، یعنی تولید نمایش یکسان برای همه تصاویر، کمک می‌کند.

```
def negative_cosine(p, z):
    z = z.detach()
    return -(normalize(p) * normalize(z)).sum(dim=1).mean()
```

۲. ساختار فولدرها

```
simsiam-project/
├── data/                  # فایل‌های CIFAR-10 در ؛ Git ذخیره نمی‌شود
├── checkpoints/           # وزن‌های آموزش‌دیده مدل
├── results/               # گزارش‌های ارزیابی و نمودارها
├── configs/config.yaml    # تنظیمات اصلی
├── models/
│   ├── encoder.py         # ConvEncoder
│   ├── projector.py       # MLP projector
│   ├── predictor.py       # predictor MLP
│   └── simsiam.py         # مدل و loss اصلی
├── datasets/
│   ├── cifar10.py         # خواندن batchهای CIFAR-10
│   └── augmentations.py   # augmentationهای tensor-based
├── training/
│   ├── pretrain.py        # self-supervised training
│   ├── linear_eval.py     # آموزش classifier خطی
│   └── supervised.py      # baseline نظارت‌شده
├── evaluation/
│   ├── metrics.py         # accuracy
│   ├── evaluate.py        # ارزیابی test set
│   └── visualization.py   # نمودارها
├── app/demo.py            # رابط Gradio محلی
├── utils/                 # seed و checkpoint
├── notebooks/experiment.ipynb
├── requirements.txt
├── README.md
└── main.py
```

نقش فایل‌های اصلی

فایل	وظیفه
models/simsiam.py	اتصال encoder، projector و predictor و محاسبه loss دوطرفه
datasets/cifar10.py	خواندن فایل‌های binary CIFAR-10 بدون نیاز به torchvision
training/pretrain.py	آموزش خودنظارتی و ذخیره simsiam.pt
training/linear_eval.py	ثابت کردن encoder و آموزش Linear(512,10)
evaluation/evaluate.py	محاسبه accuracy و accuracy هر کلاس
app/demo.py	دریافت تصویر، اجرای مدل و نمایش احتمال کلاس‌ها

۳. نصب و راه‌اندازی

ساخت محیط مجازی

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

اگر PowerShell اجازه فعال‌سازی نداد:

```
Set-ExecutionPolicy -Scope Process Bypass
```

قرار دادن داده

پوشه استخراج‌شده باید در مسیر زیر قرار گیرد:

```
data/cifar-10-batches-py/
```

و شامل data_batch_1 تا data_batch_5، فایل test_batch و batches.meta باشد.

۴. اجرای pipeline

آموزش سریع برای تست

```
python main.py train-pretrain --epochs 1 --limit 256
python main.py train-linear --epochs 1 --limit 256
python main.py train-supervised --epochs 1 --limit 256
python main.py evaluate --limit 10000
```

آموزش با تمام ۵۰,۰۰۰ تصویر

```
python main.py train-pretrain --epochs 5 --limit 50000
python main.py train-linear --epochs 5 --limit 50000
python main.py train-supervised --epochs 5 --limit 50000
python main.py evaluate --limit 10000
```

پارامتر `--limit` تعداد تصاویر آموزشی را تعیین می‌کند. روی CPU، آموزش کامل زمان‌بر است؛ برای ارائه، اجرای ۵ epoch با ۵۰,۰۰۰ تصویر انتخاب مناسبی است.

نحوه عملکرد main.py

```
python main.py train-pretrain --epochs 5 --limit 50000
```

این دستور ابتدا دیتاست را می‌خواند، DataLoader می‌سازد، دو view تولید می‌کند، loss را محاسبه می‌کند، backpropagation انجام می‌دهد و در پایان checkpoint را ذخیره می‌کند.

۵. ارزیابی و نتایج

```
python main.py evaluate --limit 10000
```

مدل	تعداد تصاویر test	Accuracy
SimSiam + Linear Classifier	۱۰,۰۰۰	۳۲٫۰۸٪
Supervised Baseline	۱۰,۰۰۰	۷۰٫۰۱٪

در supervised baseline، encoder و classifier مستقیماً با label آموزش می‌بینند؛ بنابراین در این آزمایش دقت بالاتری دارد. SimSiam با epochهای بیشتر، augmentation بهتر و تنظیم hyperparameterها می‌تواند بهبود پیدا کند.

نتایج عددی در `results/evaluation.json` ذخیره می‌شوند و checkpointها در پوشه `/checkpoints` قرار می‌گیرند.

۶. اجرای دمو

```
python main.py demo
```

پس از اجرا، مرورگر را روی آدرس زیر باز کنید:

```
http://127.0.0.1:7860
```

1. تصویر را upload کنید.
2. تصویر uploadشده نمایش داده می‌شود.
3. encoder و classifier اجرا می‌شوند.
4. کلاس پیش‌بینی‌شده و احتمال هر ۱۰ کلاس نمایش داده می‌شود.

> دمو local است و به اینترنت نیاز ندارد. تصویر گوسفند یا پشم در CIFAR-10 کلاس ندارد؛ بنابراین مدل مجبور است آن را به نزدیک‌ترین کلاس موجود نسبت دهد.

۷. Jupyter Notebook و ارائه

فایل `notebooks/experiment.ipynb` برای ارائه آماده شده است. سلول‌ها را به‌ترتیب اجرا کنید: معرفی، تست dummy batch، بارگذاری داده، checkpoint، baseline، linear evaluation، pretraining و اجرای دمو.

برای ارائه، ابتدا مسئله را توضیح دهید، سپس معماری را نشان دهید، مفهوم stop-gradient را توضیح دهید، خروجی loss و جدول accuracy را نمایش دهید و در پایان دمو را اجرا کنید.

۸. خطاهای رایج

خطا	راه‌حل
CIFAR-10 not found	مسیر `data/cifar-10-batches-py` و نام فایل‌ها را بررسی کنید.
Checkpoint not found	ابتدا pretraining و سپس linear evaluation را اجرا کنید.
Gradio import error	venv را فعال کرده و `pip install -r requirements.txt` را اجرا کنید.
آموزش کند است	`--limit` یا تعداد epoch را کاهش دهید؛ اجرای CPU طبیعی است که زمان‌بر باشد.

۹. جمع‌بندی

این پروژه یک pipeline کامل از داده تا دمو فراهم می‌کند: داده خوانده می‌شود، representation بدون label یاد گرفته می‌شود، classifier خطی آموزش می‌بیند، با baseline مقایسه می‌شود و مدل در یک رابط محلی قابل نمایش است. ساختار ماژولار باعث می‌شود هر بخش مستقل، قابل آزمایش و قابل توسعه باشد.